Stochastic Actor-Critic: Mitigating Overestimation Bias with a Single Distributional Critic

Anonymous authors
Paper under double-blind review

Abstract

Actor-critic methods in reinforcement learning train a critic with temporal-difference updates and use it as a learning signal for the policy (actor). This design typically achieves higher sample efficiency than on-policy methods. However, critic networks tend to overestimate value estimate systematically. This is often addressed by introducing a pessimistic bias based on uncertainty estimates. Many recent methods employ techniques such as ensembling to quantify the critic's epistemic uncertainty for scaling pessimistic updates. Moreover, certain network regularization methods are shown to reduce overestimation bias. In this work, we propose a new algorithm called Stochastic Actor-Critic (STAC) that incorporates only aleatoric uncertainty to scale pessimistic bias in the temporal-difference targets. STAC uses a distributional critic network to model aleatoric uncertainty in value predictions, and applies dropout to both the critic and actor networks for regularization. Our results show that pessimism based on the distributional critic alone suffices to mitigate overestimation. Introducing dropout further improves performance by means of regularization. With this combination, STAC achieves high sample efficiency using just a single distributional critic network.

1 Introduction

Actor-critic methods leverage off-policy samples to train critics, promising higher sample-efficient learning than on-policy algorithms. Despite this advantage, actor-critic agents often struggle with *overestimation bias* in value function learning due to *deadly triad* of function approximation, temporal difference, and off-policy learning (Thrun & Schwartz, 2014; Sutton & Barto, 2018; Van Hasselt et al., 2018), which destabilizes training. In this work, we revisit these issues from a fresh perspective, focusing on the use of uncertainty modeling in actor-critic architectures.

For this purpose, we introduce a novel off-policy actor-critic algorithm, Stochastic Actor-Critic (STAC), specifically designed to address both sample and computation inefficiencies. STAC is an off-policy maximum entropy distributional actor-critic algorithm employing experience buffer to sample off-policy transition tuples.

Most actor-critic methods use epistemic uncertainty for pessimism to mitigate critic overestimation, which hinders exploration of state-action space Moskovitz et al. (2021); Chen et al. (2021), while some works propose to use both epistemic and aleatoric uncertainty for this purpose Kuznetsov et al. (2020). Our contribution is that using only aleatoric uncertainty is enough to mitigate overestimation. Therefore, a single distributional critic is sufficient to capture all uncertainty sources that contribute to overestimation: transition stochasticity, the policy-induced noise in bootstrapped targets, and learning errors. By applying pessimism directly on this distribution, STAC eliminates the need for ensembles, reducing both computation and memory costs.

Our method differs from other methods using aleatoric uncertainty for safety Tang et al. (2019); Yang et al. (2021); Kim & Oh (2022). These methods, incorporate uncertainty into policy updates, whereas STAC uses it on both critic and policy updates for overestimation mitigation rather than safe learning.

Recent work of Nauman et al. (2024) has also shown that network regularization has positive impact on overestimation by reducing over-fitting. Ensemble based methods inherently regularized by using multiple

critic networks. Since STAC employs single critic network, it is sensitive to learning errors and over-fitting. For this purpose, dropout mechanism (Srivastava et al., 2014) and layer normalization (Ba et al., 2016) is employed in both actor and critic networks. In the experiments, we present controlled ablation study to isolate dropout's role: while distributional pessimism eliminates overestimation bias, dropout primarily increases training stability and performance.

The implementation is very simple and can be obtained by introducing dropout to networks and defining and pessimistic learning objective upon Distributional Soft Actor-Critic algorithm (Duan et al., 2021). We conduct experiments on standard RL benchmarks to evaluate the performance of STAC compared to existing methods. Our results demonstrate the effectiveness of STAC in achieving competitive performance to SOTA methods while requiring fewer computational resources and fewer samples, making it a promising approach for real-world RL applications.

2 Preliminaries

2.1 Aleatoric and Epistemic Uncertainty

In this part, we explain the differences between two main types of uncertainties, *aleatoric* and *epistemic* uncertainty (Der Kiureghian & Ditlevsen, 2009; Kendall & Gal, 2017; Gal et al., 2016b).

Aleatoric Uncertainty This type of uncertainty comes from the inherent randomness within the data. Even if more data are collected, it is unavoidable and cannot be reduced, because it is an intrinsic part of the process being modeled, including measurement errors or natural variability in the data.

For regression problems in deep learning setting, we can model this by having a distributional (heteroscedastic) network (with parameter θ) that outputs a probability distribution $p_\theta(y|x)$ conditioned on input x (Lakshminarayanan et al., 2017; Kendall & Gal, 2017). Given a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, the loss function for training the network can be derived from the negative log-likelihood of the normal distribution: $\mathcal{L}_\theta(\mathcal{D}) = \mathbb{E}_{\{(x_i,y_i)\}\sim\mathcal{D}}[-\log p_\theta(y_i|x_i)]$.

Epistemic Uncertainty Epistemic uncertainty reflects the uncertainty in the model parameters due to insufficient training data. Aleatoric uncertainty is high in inherently random regions while epistemic uncertainty is high where there is no (or less) data.

Given the training data $\mathcal{D}$ and prior distribution $p(\theta)$ over the network parameters θ, we can compute the posterior distribution over the parameters $p(\theta \mid \mathcal{D})$ using Bayesian inference. During prediction, we marginalize over the posterior distribution of the parameters to obtain the predictive distribution of outputs: $p(y \mid x, \mathcal{D}) = \int p(y \mid x, \theta)p(\theta \mid \mathcal{D})d\theta$, and it is often approximated by sampling several sets of parameters from the posterior distribution $p(\theta \mid \mathcal{D})$ representing epistemic uncertainty, and averaging predictions from distribution $p(y \mid x, \theta)$ representing aleatoric uncertainty.

2.2 Reinforcement Learning

This section is dedicated to briefly explain the reinforcement learning concept and actor-critic methodology. Throughout the paper, $\mathcal{P}(\Omega)$ denotes the set of all possible probability distributions on set Ω.

Model-free Reinforcement Learning In reinforcement learning language, the agent lives in a Markov Decision Process (MDP) which is represented by a tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, d_0, \tau, R)$, where $\mathcal{S}$ is state space, $\mathcal{A}$ is action space, $d_0 \in \mathcal{P}(\mathcal{S})$ is initial state distribution, $\tau : \mathcal{S} \times \mathcal{A} \to \mathcal{P}(\mathcal{S})$ is transition kernel and $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is reward function.

The initial state is sampled first, $s_0 \sim d_0(\cdot)$. At each time t being on $s_t \in \mathcal{S}$; next state is obtained from the environment, $s_{t+1} \sim \tau(\cdot \mid s_t, a_t)$ depending on the taken action $a_t \sim \pi(\cdot \mid s_t)$. Finally, a reward is obtained, $r_t = R(s_t, a_t)$ from the reward function R. The ultimate goal of the agent is to derive a policy $\pi : \mathcal{S} \to \mathcal{P}(\mathcal{A})$

to maximize discounted cumulative return, i.e., state value function for a given state s,

$$V^{\pi}(s) = \mathbb{E}_{\pi,\tau}\Big[\sum_{t=0}^{\infty}\gamma^t R(s_t,a_t)\Big|s_0=s\Big]. \tag{1}$$

Maximum Entropy Actor-Critic To promote random actions for exploration and algorithm robustness, maximum entropy framework introduces policy entropy bonus into value functions (Haarnoja et al., 2017; 2018),

$$V^{\pi}(s) = \mathbb{E}_{\pi,\tau}\Big[\sum_{t=0}^{\infty}\gamma^t R(s_t,a_t) - \alpha\log\pi(a_t|s_t)\Big|s_0=s\Big], \tag{2}$$

$$Q^{\pi}(s,a) = R(s,a) + \mathbb{E}_{\pi,\tau}\Big[\sum_{t=1}^{\infty}\gamma^t R(s_t,a_t) - \alpha\log\pi(a_t|s_t)\Big|s_0=s,a_0=a\Big]. \tag{3}$$

where Q is state-action value function. Learning iterates between solving policy evaluation and policy improvement. For the definition of state-action value function, Bellman backup operator $\mathcal{T}^{\pi}$ is defined,

$$\mathcal{T}^{\pi}Q(s,a) = R(s,a) + \gamma\mathbb{E}_{\substack{s'\sim\tau(\cdot|s,a)\\a'\sim\pi(\cdot|s')}}\big[Q(s',a') - \alpha\log\pi(a'\mid s')\big], \tag{4}$$

and the critic is expected to remain same if this operator applied on itself, i.e., $Q^{\pi}(s,a) = \mathcal{T}^{\pi}Q^{\pi}(s,a)$. Policy evaluation minimizes the temporal difference, i.e., the difference between Q and $\mathcal{T}^{\pi}Q$ to satisfy this condition. Therefore, at each iteration k, error norm is minimized to update critic,

$$Q^{k+1} = \arg\min_{Q}\mathbb{E}_{\substack{s\sim\rho_{\pi_B}\\a\sim\pi_B}}\Big[\big\|\mathcal{T}^{\pi}Q^k(s,a) - Q(s,a)\big\|\Big], \tag{5}$$

where π_B is behavioral policy and ρ_{π_B} represents distribution of visited states by π_B. In practice, the temporal difference (TD) target is used instead of the Bellman backup $\mathcal{T}^{\pi}Q(s,a)$ for learning. TD target is calculated by a random action drawn from the policy π, instead of expectation.

The optimal policy gives the maximum value, i.e., $\pi^*(\cdot|s) = \arg\max_{\pi} V^{\pi}(s)$. Policy improvement is minimizing KL divergence from policy to softmax policy on critic,

$$\pi^{k+1}(\cdot\mid s) = \mathbb{E}_{s\sim\rho_{\pi_B}}\Big[\arg\min_{\pi}\mathrm{KL}\Big(\pi(\cdot\mid s)\Big\|\frac{\exp(\alpha^{-1}Q^k(s,\cdot))}{\int_{\mathcal{A}}\exp(\alpha^{-1}Q^k(s,a))da}\Big)\Big] = \arg\max_{\pi}\mathbb{E}_{\substack{s\sim\rho_{\pi_B}\\a\sim\pi}}\Big[Q^k(s,a) - \alpha\log\pi(a|s)\Big], \tag{6}$$

After sufficient iteration, both policy and critic converge to optimality in the ideal case. In this iteration, critic is also updated, $Q^{k+1}(s,a) = \mathcal{T}^*Q^k(s,a)$, where $\mathcal{T}^*$ is the Bellman optimality opertor (Equation 5 from Haarnoja et al. (2017)),

$$\mathcal{T}^*Q(s,a) = R(s,a) + \gamma\mathbb{E}_{s'\sim\tau(\cdot|s,a)}\Big[\tilde{\alpha}\log\Big(\int_{\mathcal{A}}\exp(\tilde{\alpha}^{-1}Q(s',a'))da'\Big)\Big]. \tag{7}$$

and the optimal critic function satisfies the Bellman optimality condition, $Q^*(s,a) = \mathcal{T}^*Q^*(s,a)$. $\tilde{\alpha}\in[\alpha,\infty)$ is defined to demonstrate the continuum of policy improvement, i.e., the fact that policy is updated by partially exploiting state-action value function at each iteration. As $\tilde{\alpha}\to\infty$, $\mathcal{T}^*Q$ boils down to $\mathcal{T}^{\mathcal{U}}Q$, which represents state-action value for uniform policy. On the other hand, $\tilde{\alpha}=\alpha$ represents updated state-action value after complete exploitation.

3 Distributional Reinforcement Learning and Overestimation

3.1 Distributional Maximum Entropy Actor-Critic

Instead of learning expectation of discounted return (value function), probability distribution of return can also be modeled as a random variable to identify possible consequences of given policy (Bellemare et al., 2017). This also constitutes the methodology of Distributional Soft Actor-Critic (DSAC) algorithm (Duan et al., 2021) with maximum entropy policy. For this, state-action return distribution $\mathcal{Z} \in \mathcal{P}(\mathbb{R}^{\mathcal{S}\times\mathcal{A}})$ is defined, where state-action return is sampled from this distribution, $Z(s,a) \sim \mathcal{Z}(s,a)$. Recall that $Q(s,a) = \mathbb{E}_{\mathcal{Z},\tau}[Z(s,a)]$.

$$Z^\pi(s,a) = R(s,a) + \sum_{t=1}^{\infty} \gamma^t \big(R(s_t,a_t) - \alpha \log \pi(a_t|s_t)\big), \quad s_0 = s, \quad a_0 = a, \tag{8}$$

where $a_t \sim \pi(\cdot|s_t)$, $s_{t+1} \sim \tau(\cdot|s_t,a_t)$. Corresponding definition by Bellman backup operator $\mathcal{T}^\pi$ is defined as

$$\mathcal{T}^\pi Z(s,a) \overset{\mathrm{D}}{=} R(s,a) + \gamma\big(Z(s',a') - \alpha \log \pi(a' \mid s')\big), \quad s' \sim \tau(\cdot \mid s,a), \quad a' \sim \pi(\cdot \mid s'), \tag{9}$$

where $A \overset{\mathrm{D}}{=} B$ indicates that two random variables, A and B, have identical probability laws.

Expected Value Substitution Duan et al. (2025) proposed using expected return (value) instead of random return for bootstrapping in Bellman backup for their DSACT algortihm. This way, uncertainty due to environment stochasticity is modeled for only one step. Therefore, Bellman backup is redefined as

$$\mathcal{T}^\pi Z(s,a) \overset{\mathrm{D}}{=} R(s,a) + \gamma\big(Q(s',a') - \alpha \log \pi(a' \mid s')\big), \quad s' \sim \tau(\cdot \mid s,a), \quad a' \sim \pi(\cdot \mid s'). \tag{10}$$

To make notation easier, distributional Bellman backup $\mathcal{T}_D^\pi \mathcal{Z}$ is defined, where Bellman backup of sampled returns are sampled from this distribution, i.e., $\mathcal{T}^\pi Z(s,a) \sim \mathcal{T}_D^\pi \mathcal{Z}(s,a)$. At each step k, the state-action return distribution is updated by minimizing Kullback-Leibler divergence from distributional Bellman backup,

$$\mathcal{Z}^{k+1} = \arg\min_{\mathcal{Z}} \mathbb{E}_{\substack{s\sim\rho_{\pi_B} \\ a\sim\pi_B}} \Big[D_{\mathrm{KL}}\big(\mathcal{T}_D^\pi \mathcal{Z}^k(s,a) || \mathcal{Z}(s,a)\big)\Big]. \tag{11}$$

Policy update follows same rule shown in Equation 6 using average return (state-action value function). Policy update exploits return estimations, the Bellman optimality opertor for return distribution (similar to Equation 7) is as follows,

$$\mathcal{T}^* Z(s,a) \overset{\mathrm{D}}{=} R(s,a) + \gamma\tilde{\alpha} \log\Big(\int_{\mathcal{A}} \exp(\tilde{\alpha}^{-1} Z(s',a'))da'\Big), \quad s' \sim \tau(\cdot \mid s,a) \tag{12}$$

3.2 Quantifying Overestimation for Sub-Gaussian Return Distributions

Equation 7 and 12 has a fundamental difference. Although $Q(s,a) = \mathbb{E}_{\mathcal{Z},\tau}[Z(s,a)]$, the Bellman optimality backups are not equal, $\mathcal{T}^*Q(s,a) \neq \mathbb{E}_{\mathcal{Z},\tau}[\mathcal{T}^*Z(s,a)]$. In fact, it is difficult to calculate $\mathcal{T}^*Q(s,a)$ accurately even in non-distributional actor-critic since Q is just deterministic guess of return, which is usually not true.

Now, we analyze overestimation bias, similar to the work of Chen et al. (2021) and Lan et al. (2020) but in the soft learning framework instead of discrete actions. We define overestimation error as difference between $\mathbb{E}_{\mathcal{Z},\tau}[\mathcal{T}^*Z(s,a)]$ and average $\mathcal{T}^*Q(s,a)$ as ϵ,

$$\epsilon(s,a) = \mathbb{E}_{\mathcal{Z},\tau}[\mathcal{T}^*Z(s,a)] - \mathcal{T}^*Q(s,a). \tag{13}$$

In the ideal case, $\epsilon(s,a)$ should be zero if there is no overestimation, which is not the case due to critic uncertainty. To quantify it, we assume that critic distribution $\mathcal{Z}(s,a)$ is sub-Gaussian with variance proxy $S^2(s,a)$.

Definition 3.1. *A random variable $X \in \mathbb{R}$ with mean $\mu = \mathbb{E}[X]$ is called sub-Gaussian with variance proxy σ^2 if its moment generating function satisfies*

$$\mathbb{E}[\exp(\lambda X)] \leq \exp\left(\lambda\mu + \frac{1}{2}\lambda^2\sigma^2\right), \quad \forall\lambda \in \mathbb{R}. \tag{14}$$

Under sub-Gaussian assumption, if there exist an upper bound for overestimation, this bound can be used to devise a conservative Bellman backup operator. For this, we present Theorem 3.1. The proof is available in Appendix A.

Theorem 3.1 (Overestimation quantification for sub-Gaussian critics)**.** *Let given state-action return distribution $\mathcal{Z} \in \mathcal{P}(\mathbb{R}^{\mathcal{S}\times\mathcal{A}})$ be sub-Gaussian with mean $Q(s,a)$ and variance proxy $S^2(s,a)$ for all state-action pairs, with bounded support. Then,*

$$\mathbb{E}_{\mathcal{Z},\tau}[\mathcal{T}^* Z(s,a)] \leq R(s,a) + \gamma\mathbb{E}_{s'\sim\tau(\cdot|s,a)}\Big[\tilde{\alpha}\log\Big(\int_{\mathcal{A}} \exp(\tilde{\alpha}^{-1}Q(s',a') + \frac{1}{2}\tilde{\alpha}^{-2}S^2(s',a'))da'\Big)\Big]. \tag{15}$$

Corollary 3.1.1 (Pessimistic critic target)**.** *For policy evaluation, using pessimistic Bellman backup,*

$$\mathcal{T}^\pi_\beta Z(s,a) \overset{\text{D}}{=} R(s,a) + \gamma\big(Q(s',a') - \beta S(s',a') - \alpha\log\pi(a' \mid s')\big), \quad s' \sim \tau(\cdot \mid s,a), \quad a' \sim \pi(\cdot \mid s'), \tag{16}$$

in which shifted value estimation $\tilde{Q} = Q - \beta S$ is used instead of Q, prevents overestimation as long as $\beta \geq \max_{(s,a)} \frac{1}{2}\tilde{\alpha}^{-1}S(s,a)$.

Theorem 3.2 (Overestimation bound)**.** *Overestimation due to uncertainty of distribution $\mathcal{Z}$, denoted as ϵ, is upper bounded for Bellman updates,*

$$\epsilon(s,a) \leq \frac{\gamma}{2\tilde{\alpha}}\mathbb{E}_{s'\sim\tau(\cdot|s,a)}\Big[\max_{a'} S^2(s',a')\Big]. \tag{17}$$

The source of overestimation and necessity of pessimistic training is revealed in Theorem 3.1, Corollary 3.1.1. Moreover, maximum overesimation bound is demonstrated in Theorem 3.2. As an additional notation, we introduce pessimistic distributional Bellman backup $\mathcal{T}^\pi_{\beta,D}\mathcal{Z}$ where pessimistic Bellman backup of sampled returns are sampled from this distribution, i.e., $\mathcal{T}^\pi_\beta Z(s,a) \sim \mathcal{T}^\pi_{\beta,D}\mathcal{Z}(s,a)$. The difference between $\mathcal{T}^\pi_{\beta,D}\mathcal{Z}(s,a)$ and $\mathcal{T}^\pi_D\mathcal{Z}(s,a)$ is illustrated in Figure 1.

3.3 Optimal Pessimism Rate

Although using pessimistic critic targets for critic and policy training is not a new idea (Moskovitz et al., 2021; Kuznetsov et al., 2020; Chen et al., 2021), the question of how to determine pessimism β remains.

There is no analytical way to determine the optimum pessimism level. Moskovitz et al. (2021) focus on updating pessimism *on the fly* as a bandit problem instead of fixing it but this requires evaluating on-policy returns and introduces an online bandit to update pessimism. Cetin & Celiktutan (2023) proposed generalized pessimism learning based on expected bias between temporal difference target and base value of critic. Both of these methods use pessimism upon epistemic uncertainty as overestimation mitigation mechanism.

4 Stochastic Actor-Critic

In this section, we discuss key mechanisms needed for computation and sample efficient actor-critic learning and propose our algorithm *Stochastic Actor-Critic*. This algorithm employs *single distributional* critic network $\mathcal{Z}_\theta$ by parameter set θ which captures aleatoric uncertainty, and dropout along with layer normalization for network regularization. Policy π_ϕ outputs a `tanh` transformed normal distribution to bound actions to $[-1,1]$, and parameterized by set ϕ. Unlike other methods, STAC uses critic estimates in a pessimistic manner using only aleatoric uncertainty, for policy evaluation and policy improvement.

The rationale behind this argument is that most of the TD target randomness due to environment stochasticity and ongoing policy updates (policy improvement) are inherently appears as aleatoric uncertainty, and

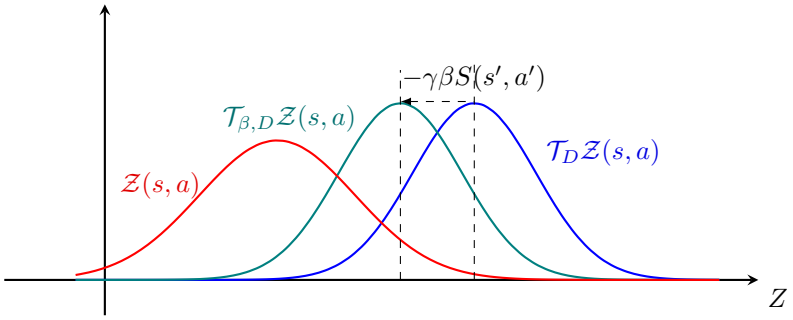

Figure 1: Evolution of Pessimistic Distributional Bellman Backup. While $\mathcal{T}_D^\pi \mathcal{Z}(s,a)$ is overestimated, a corrected backup $\mathcal{T}_{\beta,D}^\pi \mathcal{Z}(s,a)$ is closer to $\mathcal{Z}(s,a)$.

overestimation is the result of this randomness. Moreover, out-of-distribution states and actions are detected by high epistemic uncertainty, and should not be disregarded by pessimism. On the contrary, these state-action pairs should be explored with suitable methods. Therefore, in our opinion, epistemic uncertainty should not be used for overestimation mitigation unlike previous methods.

(Nauman et al., 2024) demonstrated that pessimism on critic ensemble (they call it critic regularization) degrades performance when combined with network regularization methods. Our idea is that using epistemic uncertainty for pessimism slows down learning and inherently regularizes learning. STAC directly employs network regularization by dropout and layer normalization.

4.1 Policy Evaluation

For simplicity, STAC models critic as normal distribution. This contradicts with bounded distribution assumption of Theorem 3.1, but it yields a simple loss function and is easy to interpret. Still, it is reasonable to assume critic distribution is bounded for finite horizon or discounted MDPs ($\gamma < 1$) with bounded reward functions. This assumption yields the tightest bound for overestimation, converting second inequality into an equality in the proof of Theorem 3.1. This way, overestimation is closer to the upper bound and yields the worst case.

Using transition tuples from experience replay as batch, $\mathcal{D}_b = \{(s_i, a_i, r_i, s_i', \texttt{done}_i)\}_{i=1}^{N_b}$, temporal difference (TD) target Z_i^{TD}, representing Bellman backup, is β-pessimistic,

$$Z_i^{TD} = r_i + \gamma(Q_{\bar{\theta}}(s_i', \tilde{a}_i') - \beta S_{\bar{\theta}}(s_i', \tilde{a}_i') - \alpha \log \pi_\phi(s_i', \tilde{a}_i'))(\neg\texttt{done}_i), \quad \tilde{a}_i' \sim \pi_\phi(\cdot \mid s_i'). \tag{18}$$

Learning objective is cross-entropy loss (log loss),

$$\mathcal{L}_\theta(\mathcal{D}_b) = \frac{1}{N_b}\sum_{i=1}^{N_b} -\log \mathcal{Z}_\theta(Z_i^{TD} \mid s_i, a_i) = \frac{1}{2}\log 2\pi + \frac{1}{2N_b}\sum_{i=1}^{N_b}\Big(\log S_\theta^2(s_i,a_i) + \frac{(Z_i^{TD} - Q_\theta(s_i,a_i))^2}{S_\theta^2(s_i,a_i)}\Big). \tag{19}$$

4.2 Policy Improvement

According to Corollary 3.1.1, pessimistic TD targets should be used to train critic network. However, this analysis do not account for policy learning since policy is assumed as softmax over action values. In actor-crtic framework, same pessimistic objective should be used for policy improvement, since policy evaluation objective and policy improvement objective should be the same. In other words, at learning step k, Bellman backup must be equal to Bellman policy evaluation with the new policy, $\mathcal{T}^* Z^k \overset{\text{D}}{=} \mathcal{T}^{\pi^{k+1}} Z^k$. It is only possible by using same pessimistic critic value for policy improvement.

4.3 Pessimistic Objective

In STAC, pessimism is modeled as a fixed parameter, unlike the work of (Moskovitz et al., 2021) and (Cetin & Celiktutan, 2023). There are two reasons behind this. First, introducing such a mechanism would increase complexity of the algorithm. Secondly, it is observed that a properly selected pessimism rate is enough to obtain sufficient performance, as shown in the experiments.

4.4 Dropout

STAC employs dropout to prevent over-fitting, and overestimation indirectly Nauman et al. (2024). While dropout may also promote exploration by injecting stochasticity into the policy and value estimates, we do not attempt to isolate or prove this effect here.

Dropout also allows to capture the probabilistic nature of a network, representing Bayesian neural networks (Gal & Ghahramani, 2016). Previous methods also use dropout or ensembles to approximate Bayesian posterior of critic network, for overestimation correction (Hiraoka et al., 2021; Chen et al., 2021). However, both dropout and ensemble methods regularize learning process, and STAC employs it both critic and policy networks for this purpose.

4.5 Other Details and Algorithm Summary

Layer Normalization We implement Layer Normalization (Ba et al., 2016) after all hidden activations of critic and policy networks, similar to Hiraoka et al. (2021), to regularize learning and prevent possible numerical instabilities.

Lagged critic for TD target When the trained critic network is also used in calculating the target value, the critic training is prone to divergence (Li et al., 2023). For this, a common approach is to use another critic network to evaluate TD target (Mnih et al., 2013). Similar to Lillicrap et al. (2015), Fujimoto et al. (2018), and Haarnoja et al. (2018), we use a delayed form of critic network for TD target evaluations as demonstrated in Equation 18. The parameters of target critic are only updated by Polyak averaging of main critic network weights through learning steps; $\bar{\theta} \leftarrow \rho\bar{\theta} + (1-\rho)\theta$. This strategy is important to ensure the stability of temporal difference learning.

Automatic temperature tuning Using constant temperature results in different policies if the reward magnitude changes. To mitigate this, Haarnoja et al. (2018) proposed a policy entropy constraint, representing temperature as the Lagrange multiplier of the constraint. Given target entropy $\bar{\mathcal{H}}$ as hyper-parameter, the loss function related to this constraint is as follows;

$$\mathcal{L}_\alpha(\mathcal{D}_b) = -\alpha\bar{\mathcal{H}} + \alpha \sum_{i=1}^{N_b} \mathbb{E}_{a\sim\pi_\phi(\cdot|s_i)}\big[-\log \pi_\phi(a \mid s_i)\big]. \tag{20}$$

Networks illustrations are available in Appendix D. Note that the bar notation stands for the lagged network with non-trainable parameters. STAC is summarized in Algorithm 1 with gradient descent but Adam optimizer (Kingma & Ba, 2014) is used in our experiments.

5 Experiments

Through Gymnasium API (Towers et al., 2023), six MuJoCo and three Box2D environments are used for experiments as they are tested by most algorithms in the literature. Box2D environments are inherently random, `BipedalWalker-v3` and `BipedalWalkerHardcore-v3` models a two-legged robot on randomly generated terrain, while `LunarLander-v3` models a landing spaceship under wind and turbulence, which is maximized in our experiments.

Hyper-parameters per environment can be found in Table 4 of Appendix C. For all experiments, PyTorch (version 2.7.1) (Paszke et al., 2019) is used. Please refer to Appendix E for the codebase.

Algorithm 1 Stochastic Actor-Critic

Require: Environment `env`
Require: Experience buffer $\mathcal{D}$
Require: Critic $\mathcal{Z}_\theta$, lagged critic $\mathcal{Z}_{\bar{\theta}}$, policy π_ϕ, all with dropout
Require: Initial temperature α, target entropy $\bar{\mathcal{H}}$
Require: Pessimism β
Require: Learning rates η_Q, η_π, η_α, Polyak parameter ρ
Require: Total training steps N, batch size N_b

$s \sim$ `env.reset()` ▷ Reset the environment
for N timesteps **do**
  $a \sim \pi_\phi(\cdot \mid s)$ ▷ Sample action
  $r, s', \texttt{done} \sim \texttt{env.step}(a)$ ▷ Act on environment
  $\mathcal{D} \leftarrow \mathcal{D} \cup (s, a, r, s', \texttt{done})$ ▷ Record transition tuple
  if `done` **then** $s \leftarrow s'$ **else** $s \sim$ `env.reset()` ▷ State transition or reset
  for G gradient steps **do**
    $\mathcal{D}_b = \{(s_i, a_i, r_i, s'_i, \texttt{done}_i)\}_{i=1}^{N_b} \sim \mathcal{D}$ ▷ Sample minibatch for training
    $\tilde{a}'_i \sim \pi_\phi(\cdot \mid s'_i), \quad \forall i \in \{1, 2, ..., N_b\}$ ▷ Sample next actions
    $Z_i^{TD} = r_i + \gamma(Q_{\bar{\theta}}(s'_i, \tilde{a}'_i) - \beta S_{\bar{\theta}}(s'_i, \tilde{a}'_i))(\neg \texttt{done}_i), \quad \forall i \in \{1, 2, ..., N_b\}$ ▷ Build TD targets
    $\theta \leftarrow \theta - \eta_Q \nabla_\theta \Big(\frac{1}{N_b} \sum_{i=1}^{N_b} - \log \mathcal{Z}_\theta(Z_i^{TD} \mid s_i, a_i) \Big)$ ▷ Update critic
    $\phi \leftarrow \phi - \eta_\pi \nabla_\phi \Big(\frac{1}{N_b} \sum_{i=1}^{N_b} \mathbb{E}_{a \sim \pi_\phi(\cdot \mid s_i)} \big[Q_\theta(s_i, a) - \beta S_\theta(s_i, a) - \alpha \log \pi_\phi(a \mid s_i) \big] \Big)$ ▷ Update policy
    $\alpha \leftarrow \alpha - \eta_\alpha \nabla_\alpha \Big(- \alpha \bar{\mathcal{H}} + \alpha \sum_{i=1}^{N_b} \mathbb{E}_{a \sim \pi_\phi(\cdot \mid s_i)} \big[- \log \pi_\phi(a \mid s_i) \big] \Big)$ ▷ Update temperature
    $\bar{\theta} \leftarrow \rho \bar{\theta} + (1 - \rho)\theta$ ▷ Update target critic network
  end for
end for

For evaluation, after each 1000 time steps, we execute a single test episode using the online policy and measure its performance by calculating the total reward accumulated during the episode. Specified environments are trained through a fixed number of environment interactions, repeated 5 times to assess the stability of the algorithm. Further experimental details are presented in Appendix C.

5.1 Comparison to Other Algorithms

Our experiments aim to investigate whether enhancing off-policy actor-critic methodology with STAC can improve their sample and computation efficiency on difficult continuous-control benchmarks. For this purpose, STAC is compared to similar competitive algorithms; DSACT (Duan et al., 2025) without variance-based critic gradient adjustment and SAC (Haarnoja et al., 2018). All algorithm results are obtained using in-house code with the same network architectures (including layer normalization but not dropout) to make a fair comparison.

The pessimism rate and dropout configuration used for comparison is resulted from ablation studies, and explained in the next section. Optimal configurations are summarized in Table 5.

Learning curves In Figure 2, the performance of STAC is shown against previously mentioned SOTA algorithms. Additionally, value estimation errors are presented in Figure 3. The bold lines represent the interquartile mean, while the shaded area indicates the quartiles (to represent randomness through seeds) of the total reward across evaluation episodes. Average returns through all learning processes averaged over random seeds are summarized in Table 1 and final performance is summarized in Table 2.

Sample efficiency As seen from Figure 2 and Table 1, STAC only outperforms other algorithms on many environments in terms of sample efficiency (see `BipedalWalkerHardcore-v3`, `HalfCheetah-v4`, `Hopper-v4`, `Swimmer-v4` `Walker-v4`). Although DSACT and SAC performs better on some environments, STAC still has a competitive performance, no convergence issue, and achieves this by using single critic while others use

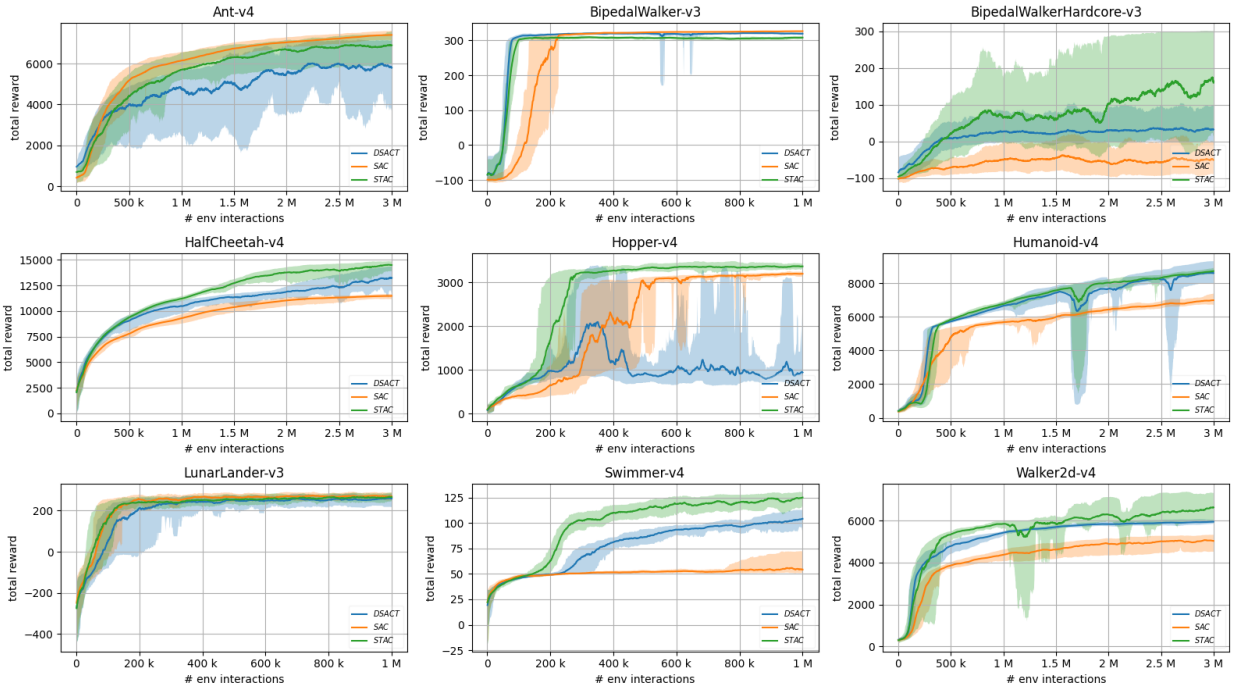

Figure 2: Episodic score curves of STAC and other algorithms.

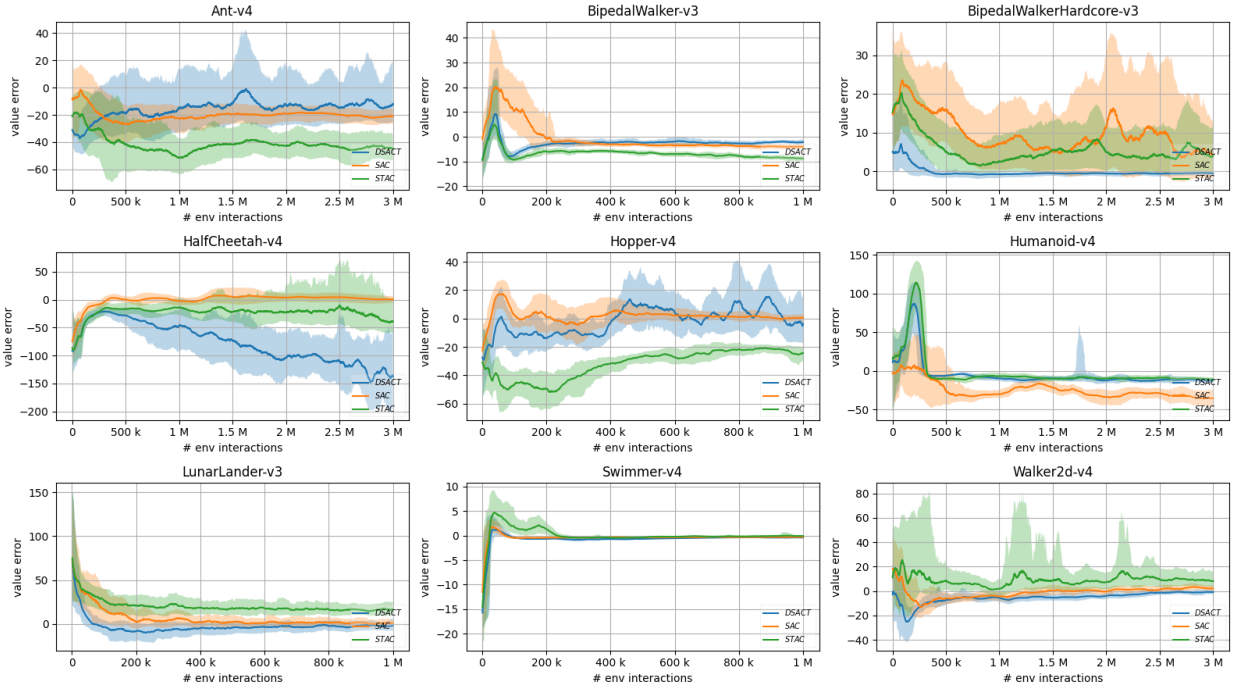

Figure 3: Average episodic value estimation error of STAC and other algorithms.

Table 1: Average episodic return through learning procedure and over five training runs on MuJoCo and Box2D tasks. ± sign represents mean and standard deviation of overall score.

Env	# steps	DSACT	SAC	STAC
Ant-v4	3M	4369.8±2289.2	**5676.6±2032.5**	5088.0±2295.8
BipedalWalker-v3	1M	**262.7±123.1**	245.7±150.2	261.5±113.3
BipedalWalkerHardcore-v3	3M	37.7±100.7	-41.7±83.3	**83.5±143.2**
HalfCheetah-v4	3M	10320.4±2640.7	9363.6±2136.8	**11291.8±3095.7**
Hopper-v4	1M	1354.0±1087.9	1998.2±1238.5	**2614.3±1167.4**
Humanoid-v4	3M	6034.8±2738.5	5179.4±2161.3	**6105.4±2739.0**
LunarLander-v3	1M	154.2±166.0	**212.6±138.2**	201.6±135.0
Swimmer-v4	1M	76.7±26.2	57.3±23.2	**97.7±32.9**
Walker2d-v4	1M	**4934.8±1596.1**	3935.1±1571.9	4829.7±2383.8

Table 2: Interquartile mean results of last %1 of evaluation episodes, β for STAC is selected with best performance, and dropout rate is 0.01.

Env	# steps	DSACT	SAC	STAC
Ant-v4	3M	5798.9	**7407.1**	6907.3
BipedalWalker-v3	1M	319.2	**326.3**	308.0
BipedalWalkerHardcore-v3	3M	32.3	-50.8	**159.9**
HalfCheetah-v4	3M	13241.6	11475.6	**14489.5**
Hopper-v4	1M	944.3	3196.4	**3363.7**
Humanoid-v4	3M	8608.9	6990.4	**8726.2**
LunarLander-v3	1M	258.3	**268.9**	265.7
Swimmer-v4	1M	104.1	54.2	**125.0**
Walker2d-v4	3M	5952.5	5036.4	**6634.7**

double. STAC has different components compared to other algorithms: pessimism is a must for convergence, while policy dropout is optional for performance improvement. However, critic dropout is necessary for some environments. Details will be discussed in the next section.

5.2 Sweeping Pessimism and Dropout

STAC is experimented by varying levels of pessimism β and dropout rates to isolate effect of pessimism and dropout individually. We used 5 pessimism level and 4 dropout configurations: no dropout, policy dropout, critic dropout and policy&critic dropout.

Learning curves Both learning curves are available in Appendix B. Episodic scores and related episodic value estimation errors for varying level of pessimism under joint policy/critic dropout are summarized in Figure 4 and Figure 5. Tabular results regarding average and final performance for different pessimism levels are summarized in Table 3. In addition, same curves for different dropout configurations under same level of pessimism are summarized in Figure 6 and Figure 7, where β values are explained in Table 5. The shaded area represents quartile limits, while the solid line represents interquartile mean across different seeds and %1 of evaluation episodes through environment steps (for smoothing).

Pessimism Results indicate that β is a sensitive parameter, the higher β yields a negative error (see Figure 5), consistent with our assumptions. In addition, score curves are worse if value estimations tend to be positive, meaning that critic overestimation is not mitigated enough (see `Ant-v4`, `BipedalWalker-v3`, `Hopper-v4`, `Humanoid-v4`, `Walker-v4`). On the other hand, score curves are again worse when error curves are negative than it should be, meaning that the learner is stuck on critic underestimation caused by high pessimism (see `BipedalWalkerHardcore-v3`, `HalfCheetah-v4`, `LunarLander-v3`, `Swimmer-v4`). In the end, pessimism sensitivity varies for different environments, possibly because of varying task difficulties. For easier tasks, less pessimism is enough but difficult tasks require it significantly. This parameter stands as the major bottleneck of STAC and can only be determined by this heuristic for now.

Table 3: Average episodic scores through learning procedure (first lines), and interquartile mean scores of last %1 of evaluation episodes (second lines), for varying β. Dropout rate is 0.01 for critic and policy.

Env	$\beta = 0.0$	$\beta = 0.125$	$\beta = 0.25$	$\beta = 0.375$	$\beta = 0.5$
Ant-v4	3181.2±2643.0	4623.6±2569.6	**5088.0±2295.8**	5084.5±1998.5	4392.7±1506.6
	5264.4	**7082.7**	6907.3	6246.4	4834.4
BipedalWalker-v3	171.5±163.3	204.3±156.7	241.5±131.8	**261.5±113.3**	249.4±124.0
	171.9	125.4	262.6	**308.0**	305.1
BipedalWalkerHardcore-v3	73.2±139.9	**78.6±138.5**	37.5±98.9	-17.1±30.5	-42.8±43.7
	61.8	**118.8**	43.4	-0.0	-59.8
HalfCheetah-v4	**11291.8±3095.7**	9815.4±2794.4	10769.5±3028.2	9938.3±2441.2	8528.9±2823.5
	14489.5	12299.6	13385.4	11042.1	7602.7
Hopper-v4	1040.9±515.2	1350.8±858.7	2107.5±1192.3	**2620.3±1130.2**	2614.3±1167.4
	1132.4	1308.8	2643.4	3318.2	**3363.7**
Humanoid-v4	3753.4±2973.4	5245.1±3211.9	**5695.8±2748.9**	5480.4±2404.1	5402.0±2387.7
	5623.9	**8407.2**	8277.9	7587.2	6930.8
LunarLander-v3	**201.6±135.0**	199.9±135.3	99.7±131.7	-23.9±100.1	-85.3±77.2
	265.7	263.8	157.2	14.3	-77.7
Swimmer-v4	**70.3±29.4**	66.4±26.8	54.5±16.1	48.2±10.5	43.4±6.0
	103.7	93.5	70.0	54.2	45.9
Walker2d-v4	3164.1±2203.5	4571.3±2146.6	4834.9±1654.7	**4949.6±1449.0**	4690.1±1361.2
	4299.2	**6306.4**	5842.1	5795.2	5539.1

Dropout Dropout is a regularization method, and expected to fix overestimation caused by over-fitting (Nauman et al., 2024). Dropout's primary role in STAC is as a regularization tool to improve optimization stability and robustness to noisy data. In Figure 6, independent of critic dropout, policy dropout increases performance in all experiments, enhancing training stability. According to Figure 7, the overestimation trends observed with and without dropout are qualitatively similar, confirming that our core contribution—mitigating overestimation via aleatoric uncertainty—remains effective independent of dropout, except `Hopper-v4`. However, in this environment, learning does not converge without critic dropout. Therefore, critic dropout is necessary in some environments for training stability rather than overestimation mitigation. Double/ensemble critic approaches mitigate training instability naturally, whereas STAC requires a convenient dropout in some cases.

6 Related Work

Pessimism upon epistemic uncertainty In the literature, the overestimation problem is mainly solved by pessimistic learning based on *epistemic uncertainty* of critic (Chen et al., 2021; Hiraoka et al., 2021). The same approach is also used in model-based RL methods (Janner et al., 2019; Chua et al., 2018; Depeweg et al., 2016). Recently, for off-policy model-free actor-critic algorithms, epistemic uncertainty is estimated by using double network (Fujimoto et al., 2018; Haarnoja et al., 2018) or ensemble network (Chen et al., 2021; Moskovitz et al., 2021). Ensemble methods are computationally expensive as there are multiple of parameters to be optimized. Using dropout is a kind of Bayesian approximation, so another way to assess epistemic uncertainty (Gal & Ghahramani, 2016). It has applications on model-based (Gal et al., 2016a; 2017; Kahn et al., 2017) and model-free (Moerland et al., 2017; Jaques et al., 2019; He et al., 2021) reinforcement learning. For actor-critic learning, Hiraoka et al. (2021) found out that using Bayesian dropout also reduces the necessary critic ensemble size. He et al. (2021) also demonstrated that a single critic network is enough when dropout is also used to evaluate Bellman backup. The mentioned methods use a constant level of pessimism for policy evaluation and improvement, while Moskovitz et al. (2021) focused on updating pessimism *on the fly* as a bandit problem rather than fixing it.

Pessimism upon aleatoric uncertainty Aleatoric uncertainty representation for the value function carries fundamental importance, especially in the presence of approximation (Bellemare et al., 2017). This can be conducted by atoms (Bellemare et al., 2017), quantiles (Dabney et al., 2018), and a probability distribution (Tang et al., 2019; Yang et al., 2021). Kuznetsov et al. (2020) claims *aleatoric uncertainty* is also responsible for overestimation since any randomness is exploited when the Bellman optimality operator ($\mathcal{T}^*$) is employed. For this purpose, they use ensemble networks for epistemic uncertainty, in which each

network is a distributional network (as quantiles) for aleatoric uncertainty assessment, and used both types of uncertainties for overestimation correction. For safety-critical RL applications to avoid catastrophic situations, pessimistic policy updates upon aleatoric uncertainty are modeled as normal distribution by Tang et al. (2019) and Yang et al. (2021). These methods use pessimism only on actor update for safety, while STAC method uses it on Bellman optimality operator (so on both actor and critic updates) to mitigate overestimation.

7 Conclusion & Future Directions

In this paper, we introduced Stochastic Actor-Critic (STAC), a novel off-policy actor-critic algorithm. The main idea is to mitigate overestimation for the sake of faster and more robust learning by incorporating the pessimistic learning objective using aleatoric uncertainty. For this, critic is modeled as a distributional (heteroscedastic) neural network. Although normal distribution is used for this purpose, our analysis is valid for all sub-Gaussian critic distributions or quantile representations. We derived an upper bound for overestimation, demonstrating that an adequate level of pessimism mitigates overestimation without succumbing to underestimation, thus facilitating computation and sample-efficient learning. Lastly, dropout for both policy and value function is utilized to enable training stability and robustness.

Adaptive Pessimism Our ablation studies demonstrate the effects of dropout rate and pessimism, revealing the sensitivity of the learning procedure to these parameters. For each specific environment and optimization method, an optimal level of pessimism and dropout exists. A promising direction for future research is to develop a grounded method to adjust the pessimism level for specific environments and agents to allow better adaptation for the learner to the environment. In addition, the sensitivity of similar algorithms to pessimism and dropout rate should be investigated in depth.

Pessimism under Stochastic Environments The effect of pessimism on highly stochastic environments is also an important topic for research. Pessimistic updates may affect exploration capability. While mitigating overestimation, adjusting risk-seeking/averse behavior is worth investigating.

Exploration Current methods use epistemic critic uncertainty for overestimation problem, which contradicts with *optimism in the face of uncertainty* principle. Future researches might consider using epistemic critic uncertainty for exploration, retaining pessimism on aleatoric critic uncertainty.

Broader Impact STAC tackles critical challenges such as accelerating learning, improving stability, and ensuring computation efficiency. Our research not only pushes the boundaries of reinforcement learning but also promises significant implications for enhancing the safety and intelligence of robots, self-driving cars, and autonomous systems in healthcare and finance.

Acknowledgments

This research has received no external funding.

References

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

Marc G Bellemare, Will Dabney, and Rémi Munos. A distributional perspective on reinforcement learning. In *International conference on machine learning*, pp. 449–458. PMLR, 2017.

Edoardo Cetin and Oya Celiktutan. Learning pessimism for reinforcement learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 37, pp. 6971–6979, 2023.

Xinyue Chen, Che Wang, Zijian Zhou, and Keith Ross. Randomized ensembled double q-learning: Learning fast without a model. *arXiv preprint arXiv:2101.05982*, 2021.

Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. *Advances in neural information processing systems*, 31, 2018.

Will Dabney, Mark Rowland, Marc Bellemare, and Rémi Munos. Distributional reinforcement learning with quantile regression. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.

Stefan Depeweg, José Miguel Hernández-Lobato, Finale Doshi-Velez, and Steffen Udluft. Learning and policy search in stochastic dynamical systems with bayesian neural networks. *arXiv preprint arXiv:1605.07127*, 2016.

Armen Der Kiureghian and Ove Ditlevsen. Aleatory or epistemic? does it matter? *Structural safety*, 31(2): 105–112, 2009.

Jingliang Duan, Yang Guan, Shengbo Eben Li, Yangang Ren, Qi Sun, and Bo Cheng. Distributional soft actor-critic: Off-policy reinforcement learning for addressing value estimation errors. *IEEE transactions on neural networks and learning systems*, 33(11):6584–6598, 2021.

Jingliang Duan, Wenxuan Wang, Liming Xiao, Jiaxin Gao, Shengbo Eben Li, Chang Liu, Ya-Qin Zhang, Bo Cheng, and Keqiang Li. Distributional soft actor-critic with three refinements. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.

Scott Fujimoto, Herke Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *International conference on machine learning*, pp. 1587–1596. PMLR, 2018.

Yarin Gal and Zoubin Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML-16)*, 2016.

Yarin Gal, Rowan McAllister, and Carl E. Rasmussen. Improving PILCO with Bayesian neural network dynamics models. In *Data-Efficient Machine Learning workshop, ICML*, April 2016a.

Yarin Gal, Jiri Hron, and Alex Kendall. Concrete dropout. *Advances in neural information processing systems*, 30, 2017.

Yarin Gal et al. Uncertainty in deep learning. 2016b.

Tuomas Haarnoja, Haoran Tang, Pieter Abbeel, and Sergey Levine. Reinforcement learning with deep energy-based policies. In *International conference on machine learning*, pp. 1352–1361. PMLR, 2017.

Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, et al. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018.

Qiang He, Huangyuan Su, Chen Gong, and Xinwen Hou. Mepg: A minimalist ensemble policy gradient framework for deep reinforcement learning. *arXiv preprint arXiv:2109.10552*, 2021.

Takuya Hiraoka, Takahisa Imagawa, Taisei Hashimoto, Takashi Onishi, and Yoshimasa Tsuruoka. Dropout q-functions for doubly efficient reinforcement learning. *arXiv preprint arXiv:2110.02034*, 2021.

Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to trust your model: Model-based policy optimization. *Advances in neural information processing systems*, 32, 2019.

Natasha Jaques, Asma Ghandeharioun, Judy Hanwen Shen, Craig Ferguson, Agata Lapedriza, Noah Jones, Shixiang Gu, and Rosalind Picard. Way off-policy batch deep reinforcement learning of implicit human preferences in dialog. *arXiv preprint arXiv:1907.00456*, 2019.

Gregory Kahn, Adam Villaflor, Vitchyr Pong, Pieter Abbeel, and Sergey Levine. Uncertainty-aware reinforcement learning for collision avoidance. *arXiv preprint arXiv:1702.01182*, 2017.

Alex Kendall and Yarin Gal. What uncertainties do we need in bayesian deep learning for computer vision? *Advances in neural information processing systems*, 30, 2017.

Dohyeong Kim and Songhwai Oh. Efficient off-policy safe reinforcement learning using trust region conditional value at risk. *IEEE Robotics and Automation Letters*, 7(3):7644–7651, 2022.

Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

Arsenii Kuznetsov, Pavel Shvechikov, Alexander Grishin, and Dmitry Vetrov. Controlling overestimation bias with truncated mixture of continuous distributional quantile critics. In *International Conference on Machine Learning*, pp. 5556–5566. PMLR, 2020.

Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. *Advances in neural information processing systems*, 30, 2017.

Qingfeng Lan, Yangchen Pan, Alona Fyshe, and Martha White. Maxmin q-learning: Controlling the estimation bias of q-learning. *arXiv preprint arXiv:2002.06487*, 2020.

Sicen Li, Qinyun Tang, Yiming Pang, Xinmeng Ma, and Gang Wang. Realistic actor-critic: A framework for balance between value overestimation and underestimation. *Frontiers in Neurorobotics*, 16:1081242, 2023.

Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.

Thomas M Moerland, Joost Broekens, and Catholijn M Jonker. Efficient exploration with double uncertain value networks. *arXiv preprint arXiv:1711.10789*, 2017.

Ted Moskovitz, Jack Parker-Holder, Aldo Pacchiano, Michael Arbel, and Michael Jordan. Tactical optimism and pessimism for deep reinforcement learning. *Advances in Neural Information Processing Systems*, 34: 12849–12863, 2021.

Michal Nauman, Michał Bortkiewicz, Piotr Miłoś, Tomasz Trzciński, Mateusz Ostaszewski, and Marek Cygan. Overestimation, overfitting, and plasticity in actor-critic: the bitter lesson of reinforcement learning. *arXiv preprint arXiv:2403.00514*, 2024.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1): 1929–1958, 2014.

Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.

Yichuan Charlie Tang, Jian Zhang, and Ruslan Salakhutdinov. Worst cases policy gradients. *arXiv preprint arXiv:1911.03618*, 2019.

Sebastian Thrun and Anton Schwartz. Issues in using function approximation for reinforcement learning. In *Proceedings of the 1993 connectionist models summer school*, pp. 255–263. Psychology Press, 2014.

Mark Towers, Jordan K. Terry, Ariel Kwiatkowski, John U. Balis, Gianluca de Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Arjun KG, Markus Krimmel, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Andrew Tan Jin Shen, and Omar G. Younis. Gymnasium, March 2023. URL `https://zenodo.org/record/8127025`.

Hado Van Hasselt, Yotam Doron, Florian Strub, Matteo Hessel, Nicolas Sonnerat, and Joseph Modayil. Deep reinforcement learning and the deadly triad. *arXiv preprint arXiv:1812.02648*, 2018.

Qisong Yang, Thiago D Simão, Simon H Tindemans, and Matthijs TJ Spaan. Wcsac: Worst-case soft actor critic for safety-constrained reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pp. 10639–10646, 2021.

Appendix A Proofs

Proof of Theorem 3.1. Analyzing expected Bellman update $\mathbb{E}_{\tau,\mathcal{Z}}[\mathcal{T}^*Z(s,a)]$,

$$\begin{aligned}
\mathbb{E}_{\tau,\mathcal{Z}}[\mathcal{T}^*Z(s,a)] &= R(s,a) + \gamma \mathbb{E}_{s'\sim\tau(\cdot|s,a)}\Big[\mathbb{E}_{Z\sim\mathcal{Z}}\Big[\alpha \log\Big(\int_{\mathcal{A}} \exp(\alpha^{-1}Z(s',a'))da'\Big)\Big]\Big] \\
&\leq R(s,a) + \gamma \mathbb{E}_{s'\sim\tau(\cdot|s,a)}\Big[\alpha \log\Big(\mathbb{E}_{Z\sim\mathcal{Z}}\Big[\int_{\mathcal{A}} \exp(\alpha^{-1}Z(s',a'))da'\Big]\Big)\Big] \\
&= R(s,a) + \gamma \mathbb{E}_{s'\sim\tau(\cdot|s,a)}\Big[\alpha \log\Big(\int_{\mathcal{A}} \mathbb{E}_{Z\sim\mathcal{Z}}\Big[\exp(\alpha^{-1}Z(s',a'))\Big]da'\Big)\Big] \\
&\leq R(s,a) + \gamma \mathbb{E}_{s'\sim\tau(\cdot|s,a)}\Big[\alpha \log\Big(\int_{\mathcal{A}} \exp(\alpha^{-1}Q(s',a') + \frac{1}{2}\alpha^{-2}S^2(s',a'))da'\Big)\Big]
\end{aligned}$$

First inequality comes from Jensen's inequality (using concave property of log function) while the following equality is a result of Tonelli's theorem. The second inequality results from the property of sub-Gaussian distribution 3.1. □

Proof of Corollary 3.1.1. From the Theorem 3.1, we can show that

$$\begin{aligned}
\mathbb{E}_{Q\sim\mathcal{Q}}[\mathcal{T}^*Q(s,a)] &\leq R(s,a) + \gamma \mathbb{E}_{s'\sim\tau(\cdot|s,a)}\Big[\alpha \log\Big(\int_{\mathcal{A}} \exp(\alpha^{-1}(Q(s',a') - \beta S(s',a') + \frac{1}{2}\alpha^{-1}S^2(s',a')))da'\Big)\Big] \\
&= R(s,a) + \gamma \mathbb{E}_{s'\sim\tau(\cdot|s,a)}\Big[\alpha \log\Big(\int_{\mathcal{A}} \exp(\alpha^{-1}Q^\dagger(s',a'))da'\Big)\Big] = \mathcal{T}^*Q^\dagger(s,a).
\end{aligned}$$

where we have defined $Q^\dagger(s',a') = Q(s',a') - \beta S(s',a') + \frac{1}{2}\alpha^{-1}S^2(s',a'))$. If $\beta \geq \max_{(s',a')} \frac{1}{2}\alpha^{-1}S(s',a')$, then $Q^\dagger(s',a') < Q(s',a')$. So we can show that

$$\mathbb{E}_{Q\sim\mathcal{Q}}[\mathcal{T}^*Q(s,a)] \leq \mathcal{T}^*Q^\dagger(s,a) \leq \mathcal{T}^*Q(s,a). \tag{21}$$

□

Proof of Theorem 3.2. From the Theorem 3.1, we can show that

$$\begin{aligned}
\mathbb{E}_{\tau,\mathcal{Z}}[\mathcal{T}^*Z(s,a)] &\leq R(s,a) + \gamma \mathbb{E}_{s'\sim\tau(\cdot|s,a)}\Big[\alpha \log\Big(\int_{\mathcal{A}} \exp(\alpha^{-1}Q(s',a') + \frac{1}{2}\alpha^{-2}S^2(s',a'))da'\Big)\Big] \\
&\leq R(s,a) + \gamma \mathbb{E}_{s'\sim\tau(\cdot|s,a)}\Big[\alpha \log\Big(\big(\int_{\mathcal{A}} \exp(\alpha^{-1}Q(s',a'))da'\big) \cdot \big(\max_{a'} \exp(\frac{1}{2}\alpha^{-2}S^2(s',a'))\big)\Big)\Big] \\
&= R(s,a) + \gamma \mathbb{E}_{s'\sim\tau(\cdot|s,a)}\Big[\alpha \log\Big(\int_{\mathcal{A}} \exp(\alpha^{-1}Q(s',a'))da'\Big) + \frac{1}{2\alpha}\max_{a'} S^2(s',a')\Big] \\
&= R(s,a) + \gamma \mathbb{E}_{s'\sim\tau(\cdot|s,a)}\Big[\alpha \log\Big(\int_{\mathcal{A}} \exp(\alpha^{-1}Q(s',a'))da'\Big)\Big] + \frac{\gamma}{2\alpha}\mathbb{E}_{s'\sim\tau(\cdot|s,a)}\Big[\max_{a'} S^2(s',a')\Big].
\end{aligned}$$

The following inequality is a result of the mean value theorem for integrals. In the last equality, the first two terms are equal to $\mathcal{T}^*Q(s,a)$. Therefore,

$$\epsilon(s,a) = \mathbb{E}_{Q\sim\mathcal{Q}}[\mathcal{T}^*Q(s,a)] - \mathcal{T}^*Q(s,a) \leq \frac{\gamma}{2\alpha}\mathbb{E}_{s'\sim\tau(\cdot|s,a)}\Big[\max_{a'} S^2(s',a')\Big].$$

□

Appendix B Results of Ablation Studies

B.1 Pessimism Ablation

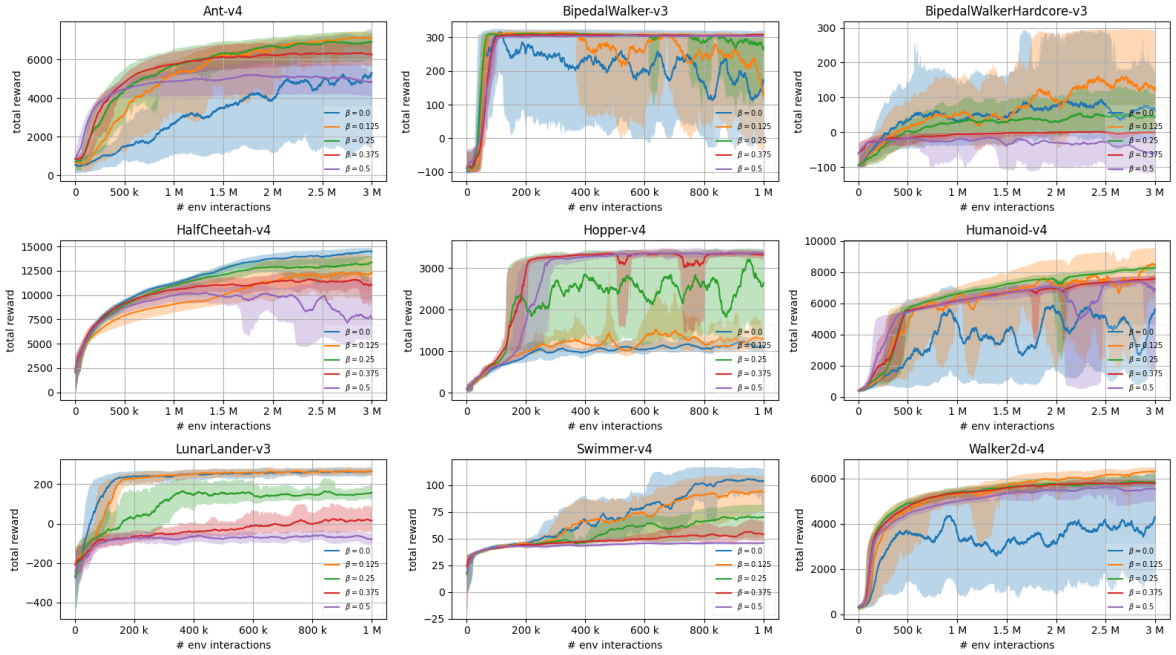

Figure 4: Episodic score curves of STAC with varying pessimism (β) parameter, with policy and critic dropout equal to 0.01.

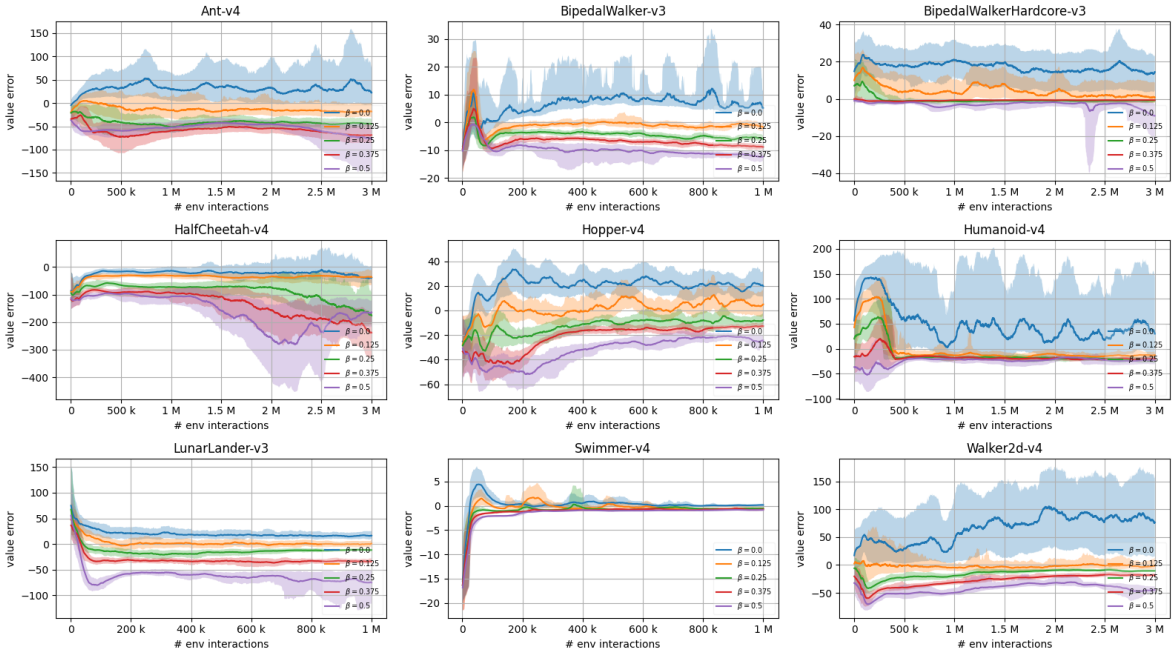

Figure 5: Average episodic value estimation error curves of STAC with varying pessimism (β) parameter, with policy and critic dropout equal to 0.01.

B.2 Dropout Configuration Ablation

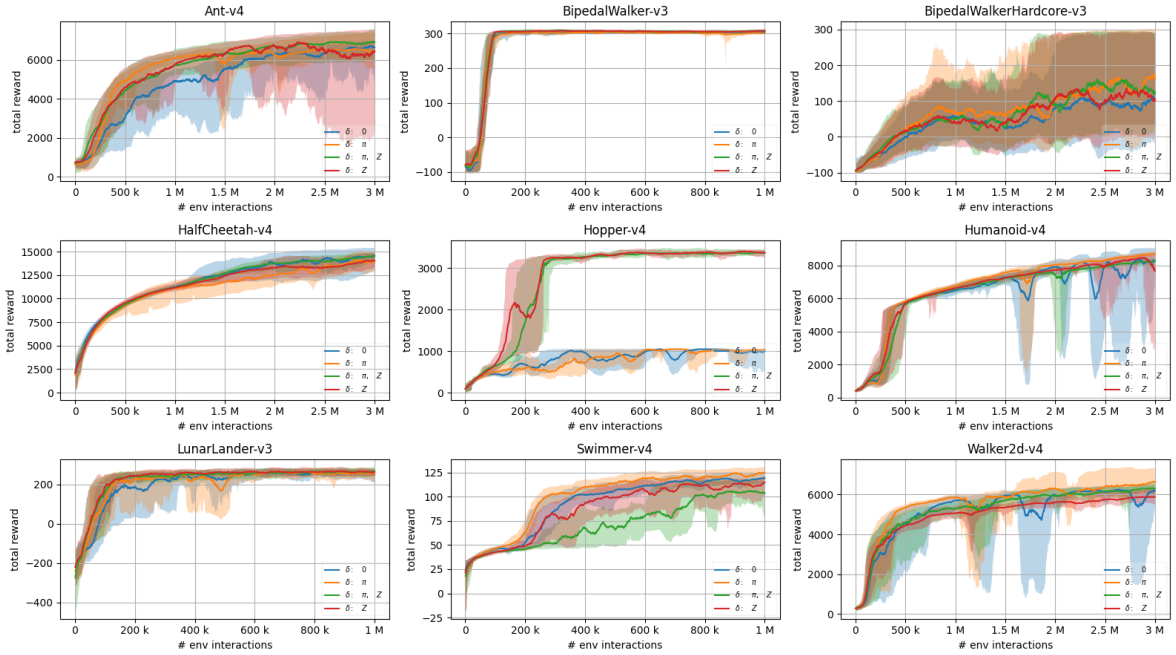

Figure 6: Episodic score curves of STAC with 4 different dropout configurations, under fixed pessimism level yielding best performance (see Table 5).

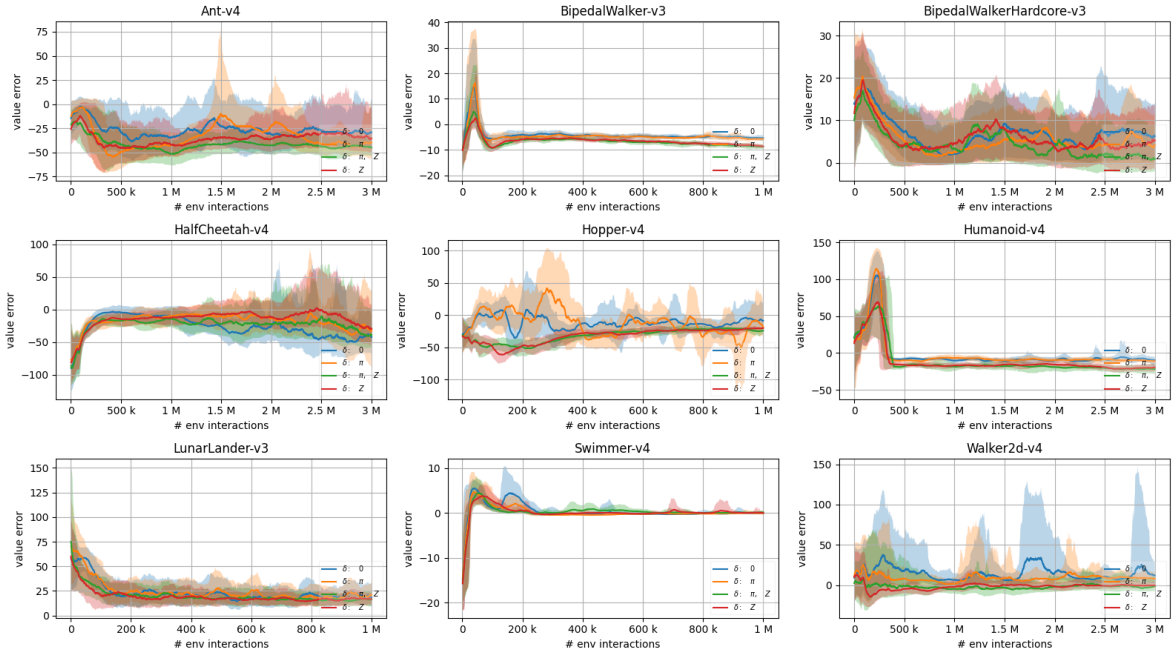

Figure 7: Average episodic value estimation error curves of STAC with 4 different dropout configurations, under fixed pessimism level yielding best performance (see Table 5).

Appendix C Hyper-parameters and Experiment Details

Table 4: Experimental Parameters per Algorithm

Algorithm	Parameter	Value
SAC, DSACT, STAC	Optimizer	Adam ((Kingma & Ba, 2014))
	Critic Learning Rate	3×10^{-4}
	Actor Learning Rate	3×10^{-4}
	Discount Rate (γ)	0.99
	Target-Smoothing Coefficient (ρ)	0.995
	Target Entropy ($\bar{H}$)	$-\lvert\mathcal{A}\rvert$
	Replay Buffer Size	1×10^{6}
	Mini-Batch Size	256
	Random Starting Data	10000
	UTD Ratio (G)	1

Table 5: Target policy entropy ($\bar{H}$), pessimism β and dropout rates per environment, yielding best results. SAC and DSACT shares same policy entropies.

Environment	Entropy ($\bar{H}$)	Pessimism (β)	Policy Dropout	Critic Dropout
`Ant-v4`	-4	0.25	0.01	0.01
`BipedalWalker-v3`	-2	0.375	0.01	0.01
`BipedalWalkerHardcore-v3`	-2	0.125	0.01	0
`HalfCheetah-v4`	-3	0.0	0.01	0.01
`Hopper-v4`	-1	0.5	0.01	0.01
`Humanoid-v4`	-8	0.25	0.01	0
`LunarLander-v3`	-2	0.0	0.01	0.01
`Swimmer-v4`	-1	0.0	0.01	0
`Walker2d-v4`	-3	0.125	0.01	0

Appendix D Network Architectures of STAC

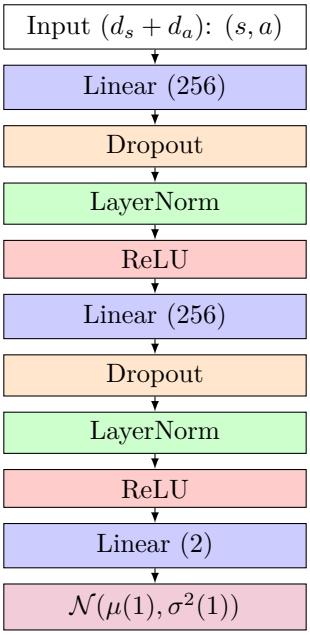

Figure 8: Critic network architecture

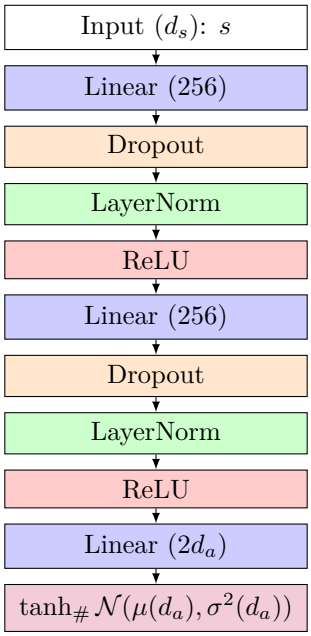

Figure 9: Policy network architecture

Appendix E Source Code

Our results can be accessed publicly at `https://github.com/authors-github/stochastic-actor-critic-results`. This code uses our in-house developed RL framework as a sub-repository, available on `https://github.com/authors-github/rl-warehouse`.